

Security Research Laboratory : Struts 2 Remote ...

Security Research Laboratory : Struts 2 Remote ... - Author not stated, communities.coverity.com.

- Published: date not stated
- Original: <<http://web.archive.org/web/20160507023636/https://communities.coverity.com/blogs/security/2013/05/29/struts2-remote-code-execution-via-ognl-injection>>
- Current location: <<https://communities.coverity.com/blogs/security/2013/05/29/struts2-remote-code-execution-via-ognl-injection>>
- Preserved from: <https://communities.coverity.com/blogs/security/2013/05/29/struts2-remote-code-execution-via-ognl-injection> (stored) on 2026-08-09
- Capture timestamp: 20130823042415
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Advisory

Overview

(Note: this write-up uses the [Maven](#) sample application provided by Struts2. Refer to the Appendix section at the bottom to install the application. References to the blank-archetype application refer to this sample application.)

From the Struts 2 [website](#)

Apache Struts 2 is an elegant, extensible framework for creating enterprise-ready Java web applications. The framework is designed to streamline the full development cycle, from building, to deploying, to maintaining applications over time.

Struts 2 heavily utilizes [OGNL](#) as a templating / expression language. OGNL, similar to other expression languages, is vulnerable to a class of issues informally termed "double evaluation". That is, the value of an OGNL expression is mistakenly evaluated again as an OGNL expression. For a background on previous OGNL double evaluation issues, I recommend [@meder's "Milking a hors](#)

e or executing remote code in modern Java frameworks" presentation. (The exploit used below is based on @meder's exploit, just condensed.) For other examples of double evaluation in different expression languages, check out Aspect Security's "Remote Code with Expression Language".

Struts 2 calls its controllers [Actions](#). Actions are mapped to URLs and views within an [XML configuration file](#) or via Java annotations. For a good background on Struts 2 and `Actions`, refer to their "Getting Started" page.

Struts 2 allows a developer to configure [wildcard mappings](#) in its XML configuration files. The blank-archetype application has the following wildcard example in its XML configuration:

```
<action name="*" class="tutorial2.example.ExampleSupport">
  <result>/example/{1}.jsp</result>
</action>
```

This allows one to specify an arbitrary Action name. If the name doesn't match any of the other more specific mappings in the XML configuration (or possibly others annotated in the Java code), then this acts as a catch-all. The Action name provided is substituted as a component of the file name. Struts then dispatches to the selfsame JSP defined in the [result](#) section.

Vulnerability and Exploit

There exists a vulnerability in this Action name to replacement mapping. If the Action name provided is in the form of `${STUFF_HERE}` or `%{STUFF_HERE}`, and the contents of the expression are [OGNL](#), then Struts2 unsafely double evaluates the contents.

To view this exploit, start up the blank-archetype application using `jetty:run`. The [following URL](#) exploits a vulnerability within the replacement support in Struts 2. If the exploit is successful, something similar to the following should be displayed:

HTTP ERROR 404

Problem accessing /struts2-blank/example/0.jsp. Reason:

Not Found

Note the "0.jsp" part in the 404 page. When successfully executed, `Process.waitFor()` returns a value of "0". This is then used as the JSP file name, "0.jsp". This implies the `touch aaa` executed successfully. A patched version doesn't have a return value since the process never executed.

Root Cause Analysis

Using [JavaSnoop](#), instrumenting the blank-archetype application application, and setting canaries for strings to match against the payload URL showed numerous potential traces. Scoping the trace to `org.apache.struts2` packages shows an interesting call to [StrutsResultSupport conditionalParse](#):

```

/**
 * Parses the parameter for OGNL expressions against the valuestack
 *
 * @param param The parameter value
 * @param invocation The action invocation instance
 * @return The resulting string
 */
protected String conditionalParse(String param, ActionInvocation invocation) {
    if (param != null && invocation != null) {
        return TextParseUtil.translateVariables(param, invocation.getStack(),
            new TextParseUtil.ParsedValueEvaluator() {
                public Object evaluate(String parsedValue) {
                    if (encode) {
                        if (parsedValue != null) {
                            try {
                                // use UTF-8 as this is the recommended encoding by W3C to
                                // avoid incompatibilities.
                                return URLEncoder.encode(parsedValue, "UTF-8");
                            }
                            catch(UnsupportedEncodingException e) {
                                if (LOG.isWarnEnabled()) {
                                    LOG.warn("error while trying to encode [" + parsedValue + "]", e);
                                }
                            }
                        }
                    }
                    return parsedValue;
                }
            });
    } else {
        return param;
    }
}

```

The method above is called from the `StrutsResultSupport.execute(ActionInvocation)`. It then calls `TextParseUtil.translateVariables`:

```

/**
 * Function similarly as {@link #translateVariables(char, String, ValueStack)}
 * except for the introduction of an additional evaluator that allows
 * the parsed value to be evaluated by the evaluator. The evaluator
 * could be null, if it is it will just be skipped as if it is just calling
 * {@link #translateVariables(char, String, ValueStack)}.
 *
 * <p/>
 *
 * A typical use-case would be when we need to URL Encode the parsed value. To do so
 * we could just supply a URLEncodingEvaluator for example.
 *
 * @param expression
 * @param stack
 * @param evaluator The parsed Value evaluator (could be null).
 * @return the parsed (and possibly evaluated) variable String.
 */
public static String translateVariables(String expression, ValueStack stack, ParsedValueEvaluator evaluator) {
    return translateVariables(new char[]{'$', '%'}, expression, stack, String.class, evaluator).toString();
}

```

This method evaluates expressions surrounded with `${}` or `%{}`. The subsequent call to `translateVariables` method evaluates the expression via the `parser.evaluate` call:

```

/**
 * Converted object from variable translation.
 *
 * @param open
 * @param expression
 * @param stack
 * @param asType
 * @param evaluator
 * @return Converted object from variable translation.
 */
public static Object translateVariables(char[] openChars, String expression, final ValueStack stack, final Class asType, final
    ParsedValueEvaluator ognlEval = new ParsedValueEvaluator() {
        public Object evaluate(String parsedValue) {
            Object o = stack.findValue(parsedValue, asType);
            if (evaluator != null && o != null) {
                o = evaluator.evaluate(o.toString());
            }
            return o;
        }
    };
    TextParser parser = ((Container)stack.getContext()).get(ActionContext.CONTAINER).getInstance(TextParser.class);
    XWorkConverter conv = ((Container)stack.getContext()).get(ActionContext.CONTAINER).getInstance(XWorkConverter.class);
    Object result = parser.evaluate(openChars, expression, ognlEval, maxLoopCount);
    return conv.convertValue(stack.getContext(), result, asType);
}

```

That passes the expression to an instance of `OgnlTextParser.evaluate`. And then it's game over.

Other Vectors

Suspicious calls to `TextParseUtil.translateVariables` were also examined for exploitability.

org.apache.struts2.dispatcher.HttpHeaderResult.execute (Tested)

`HttpHeaderResult.execute` has the following call to `TextParseUtil.translateVariables`:

```

if (headers != null) {
    for (Map.Entry<String, String> entry : headers.entrySet()) {
        String value = entry.getValue();
        String finalValue = parse ? TextParseUtil.translateVariables(value, stack) : value;
        response.addHeader(entry.getKey(), finalValue);
    }
}

```

The blank-archetype application's HelloWorld XML example was modified below to test out the call. This is probably a very unlikely scenario and also can be mitigated by the `<param name="parse">false</param>` setting. (By default, this value is true.) In this case, the `${message}` value is user-controllable within the `HelloWorld` class. This tainted value is then supplied as a header. While it's an obvious header injection, it's also a RCE vector.

```

<action name="HelloWorld" class="com.coverity.internal.examples.example.HelloWorld">
    <result name="success">/example/HelloWorld.jsp</result>
    <result name="foobar" type="httpheader">
        <param name="headers.foobar">${message}</param>
    </result>
</action>

```

org.apache.struts2.views.util.DefaultUrlHelper.* (Tested)

Pretty much every method in the `DefaultUrlHelper` class allows for RCE if one of the parameters is tainted. This is because of the `DefaultUrlHelper.translateVariable` method is called by most methods in the class. This class is also heavily utilized throughout Struts2 as the default `UrlHelper` class via `s truts-default.xml`.

Here is an instance of the defect, mocked up from the blank-archetype application `HelloWorld.jsp`:

```

<s:url id="url" action="HelloWorld">
    <s:param name="request_locale"><s:property value="message"/></s:param>
</s:url>

```

Assume the `s:property 'message'` is tainted via `?message=${OGNL_HERE}`. Since the `s:url` / URL component uses the `DefaultUrlHandler.urlRenderer` (via `ServletUrlRenderer`), the parameter is double evaluated as OGNL.

org.apache.struts2.util.URLBean.getURL (Untested)

`URLBean` seems to mainly be used in Velocity via a macro. If `URLBean` is called w/o a `setPage()` method and either the `addParameter()` method contains tainted data or no `addParameter()` method

calls occur, then URLBean seems susceptible to RCE via the DefaultUrlHelper issue above.

Non-Vectors

Some potential vectors that seemed to double evaluate OGNL were tested but found not to be exploitable using this technique.

When the action name is used as a replacement value within the `method` attribute of the Action, the replacement value is not double evaluated. Rather, the unevaluated value is passed as a method name via reflection. The blank-archetype application has this example, which is not exploitable:

```
<action name="Login_*" method="{1}" class="tutorial.example.Login">
  <result name="input">/example/Login.jsp</result>
  <result type="redirectAction">Menu</result>
</action>
```

Another non-vector tested in the blank-archetype application is to enable [Dynamic Method Invocation](#). Then modify HelloWorld Action mapping in the blank-archetype application as follows:

```
<action name="HelloWorld" class="tutorial.example.HelloWorld">
  <result>/example/${message}.jsp</result>
</action>
```

Finally, call the `getMessage` function on the HelloWorld Action (`HelloWorld!getMessage?message=${PAYLOAD}`). Test stack traces didn't show the OGNL expression being evaluated twice.

Untested

[Struts2 annotations](#) may be susceptible but have not been tested.

Testing

Outside of testing for vulnerable versions of Struts 2, testers can use a blind-ish dynamic technique:

- Identify Actions (usually via a `.action` suffix) and fingerprint responses to the Actions. For this example URL `[http://www.example.com/app/Bar.action](http://www.example.com/app/Bar.action)` , the Action name is `Bar` .
- For each Action, substitute the Action name with `ACTION_NAME` in the following expression: `$_7B%23foo='ACTION_NAME',%23foo%7D` . For example: `$_7B%23foo='Bar',%23foo%7D` .
- Replace the Action name in the URL with the substituted expression. For example:
`[http://www.example.com/app/$_7B%23foo='Bar',%23foo%7D.action]`
`(http://www.example.com/app/$_7B%23foo='Bar',%23foo%7D.action)` .

- If the Action is susceptible to this double evaluation vector, the application ought to return the same page as before. If it's not vulnerable, a 404 or other page will probably be returned.

To test out the `Welcome.action` blank-archetype above via `jetty:run`, use [this URL](#).

Remedy

Struts developers recommend upgrading to 2.3.14.2. Refer to [S2-013](#) and [S2-014](#) for details. The Struts developers mitigate the effects of double evaluation. While double evaluation still occurs within the sample application, remote code execution is not possible using @meder's vector.

Maven Appendix

First, [get Maven](#). Then create an application based on the [blank-archetype](#).

```
mvn archetype:generate -B -DgroupId=tutorial -DartifactId=tutorial -DarchetypeGroupId=org.apache.struts -DarchetypeId=tutorial # ensure the struts2 entry in the pom.xml points to 2.3.14
mvn package jetty:run
```

To test out the application, try accessing the `Welcome` Action by navigating to [following URL](#).

Archived from <http://web.archive.org/web/20160507023636/https://communities.coverity.com/blogs/security/2013/05/29/struts2-remote-code-execution-via-ognl-injection>. Rendered offline from the local Markdown copy.